Semester administration console: term scheduling, academic calendars, and role-based permissions in one dashboard

Computer
Show the registrar academic calendar workflows and approval state machine
Detail the role-based permission matrix across registrar and admin roles
Map out the data model and schema for terms sections and rooms
Show the UI wireframe spec for the department scheduling workspace

Semester's registrar dashboard should unify **term setup, calendar governance, schedule-of-classes workflows, room coordination, and permission-scoped approvals** in one operator workspace. Registrar scheduling practices at universities commonly rely on department schedulers, standard meeting patterns, room conflict detection, and workflow queues for changes that need special review. [1] [2] [3]

Registrar calendar workflow

Academic calendar and schedule publication should run as a controlled lifecycle rather than ad hoc edits, because schedule tools are meant to surface conflicts early, centralize updates, and reduce publishing errors. [4]

Workflow stages

```
Academic Year Planning
  → Term Drafting
  → Calendar Review
  → Department Schedule Build
  → Conflict Validation
  → Registrar Review
  → Room Assignment / Exception Review
  → Publish Schedule
  → Refine Window
  → Locked / Official
  → Archive
```

State machine

State	Owner	Allowed actions
Draft	Registrar admin	Create term, sessions, deadlines, time-pattern rules
In review	Registrar + academic affairs	Comment, request edits, approve policy dates
Open for department build	Department schedulers	Propose sections, meeting patterns, room requests
Validation failed	Department schedulers + registrar	Resolve conflicts, fix missing fields
Registrar review	Registrar office	Approve, return, escalate, hold
Room coordination	Registrar/space team	Assign central rooms, review exceptions
Published	Registrar	Release schedule to student-facing systems
Refine mode	Departments with scoped rights	Submit limited post-publish changes
Locked	Registrar only	Emergency edits and official corrections only
Archived	System + registrar	Read-only historical term

Approval transitions

```
Draft
  → In review
  → Open for department build
  → Validation failed
  → Registrar review
  → Room coordination
  → Published
  → Refine mode
  → Locked
  → Archived
```

Special review queues are a realistic requirement because registrar scheduling tools often route certain updates into workflow queues and limit who can request post-publish changes. [3]

Approval rules

Key rules should include:

- A term cannot open for department build until standard time patterns, milestone dates, and publication targets are configured.
- A section cannot move to registrar review if required fields are missing.
- A central-room request cannot finalize if room, instructor, or time conflicts remain unresolved.
- Post-publish edits should be limited to designated scheduler roles and often require workflow review. Universities commonly restrict change requests after publish dates and rely on designated departmental contacts. [3]

- Official publication should create a version snapshot for audit and rollback comparison.

Role-based permission matrix

Education RBAC should define what each role can view, edit, export, and administer, with logs for permission-sensitive actions. FERPA-oriented systems also emphasize scoped access rather than all-or-nothing administrative visibility. [5] [6] [7]

Registrar/admin role set

Role	Scope	Core purpose
Super admin	Tenant-wide	Platform/security administration
Registrar admin	Institution or campus	Owns official academic calendar and schedule publication
Registrar scheduler	Institution/campus/college	Reviews schedule changes, room coordination, compliance
Department scheduler	Department/program	Builds and maintains sections for assigned units
Department chair/dean reviewer	Department/college	Approves departmental submissions and exceptions
Room/space manager	Central or departmental room pools	Assigns rooms, manages capacity/features/conflicts
Academic affairs reviewer	Institution/college	Policy and curricular oversight
Read-only auditor	Scoped	Audit and review without edit power

Permission matrix

Capability	Super admin	Registrar admin	Registrar scheduler	Department scheduler	Chair/dean reviewer	Room manager	Auditor
Create academic year/term	Yes	Yes	No	No	No	No	No
Edit academic calendar milestones	Yes	Yes	Limited	No	Review only	No	No
Configure standard time patterns	Yes	Yes	Yes	No	No	No	No
Create sections	Yes	Yes	Yes	Yes	No	No	No
Edit department sections before publish	Yes	Yes	Yes	Yes	Limited	No	No

Capability	Super admin	Registrar admin	Registrar scheduler	Department scheduler	Chair/dean reviewer	Room manager	Auditor
Submit department schedule	Yes	Yes	Yes	Yes	No	No	No
Return schedule for edits	Yes	Yes	Yes	No	Yes	No	No
Approve for publication	Yes	Yes	Limited	No	Limited	No	No
Assign central rooms	Yes	Yes	Yes	Request only	No	Yes	No
Assign department rooms	Yes	Yes	Yes	Yes, if scoped	No	Yes	No
Override conflict rules	Yes	Yes	Limited with reason	No	No	Limited with reason	No
Publish official schedule	Yes	Yes	No	No	No	No	No
Open refine mode	Yes	Yes	Yes	No	No	No	No
Request post-publish change	Yes	Yes	Yes	Yes	Yes	Yes	No
Export full schedule data	Yes	Yes	Yes	Scoped only	Scoped only	Scoped only	Scoped only
View FERPA audit logs	Yes	Limited	Limited	No	No	No	Yes

Permission design principles

- Separate **view**, **edit**, **approve**, **publish**, **export**, and **override** permissions.
- Scope all rights by institution, campus, college, department, room pool, or term.
- Require justification for conflict overrides, exports, and post-lock changes.
- Time-limit elevated permissions.
- Log all record access and all sensitive administrative actions. FERPA guidance stresses reasonable controls over access to education records.[7]

Data model and schema

The academic scheduling domain should model terms, calendar events, sections, meetings, instructors, and rooms as separate but linked entities. Standardized time-block policies and room coordination practices at universities make these distinctions operationally important.[8] [9]

Core entities

Entity	Purpose
`academic_years`	Top-level yearly container
`terms`	Semester/quarter/session container
`parts_of_term`	Subterms such as first-half, summer sessions
`academic_calendar_events`	Registration, holidays, exams, deadlines
`time_patterns`	Approved meeting templates and standard blocks
`courses`	Catalog course identity
`sections`	Scheduled class offerings
`section_meetings`	Meeting days/times/locations
`section_instructors`	Assigned faculty or staff
`rooms`	Physical teaching spaces
`room_features`	Capacity, AV, accessibility, furniture
`room_pools`	Central vs department-controlled room inventories
`schedule_change_requests`	Controlled edits and review history
`workflow_actions`	Approvals, comments, returns, escalations

Example relational schema

```
CREATE TABLE academic_years (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  starts_on date NOT NULL,
  ends_on date NOT NULL,
  status text NOT NULL CHECK (status IN ('draft','active','archived'))
);

CREATE TABLE terms (
  id uuid PRIMARY KEY,
  academic_year_id uuid NOT NULL REFERENCES academic_years(id),
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  term_type text NOT NULL CHECK (term_type IN ('semester','quarter','summer','winter
  starts_on date NOT NULL,
  ends_on date NOT NULL,
  publish_at timestamptz,
  state text NOT NULL CHECK (
    state IN (
      'draft',
      'in_review',
      'open_for_department_build',
      'validation_failed',
```

```
      'registrar_review',
      'room_coordination',
      'published',
      'refine_mode',
      'locked',
      'archived'
    )
  )
);

CREATE TABLE parts_of_term (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL REFERENCES terms(id),
  code text NOT NULL,
  name text NOT NULL,
  starts_on date NOT NULL,
  ends_on date NOT NULL,
  UNIQUE(term_id, code)
);

CREATE TABLE academic_calendar_events (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL REFERENCES terms(id),
  event_type text NOT NULL,
  title text NOT NULL,
  starts_at timestamptz NOT NULL,
  ends_at timestamptz NOT NULL,
  audience_scope text NOT NULL,
  is_publishable boolean NOT NULL DEFAULT true,
  approval_state text NOT NULL CHECK (approval_state IN ('draft','pending','approved
);

CREATE TABLE time_patterns (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL REFERENCES terms(id),
  code text NOT NULL,
  label text NOT NULL,
  days_mask text NOT NULL,
  start_time time NOT NULL,
  end_time time NOT NULL,
  minutes_per_week integer NOT NULL,
  allowed_for_central_rooms boolean NOT NULL DEFAULT true,
  UNIQUE(term_id, code)
);

CREATE TABLE courses (
  id uuid PRIMARY KEY,
  subject_code text NOT NULL,
  catalog_number text NOT NULL,
  title text NOT NULL,
  credit_hours numeric(4,2) NOT NULL,
  owning_org_unit_id uuid NOT NULL,
  UNIQUE(subject_code, catalog_number)
);

CREATE TABLE sections (
```

```
  id uuid PRIMARY KEY,
  course_id uuid NOT NULL REFERENCES courses(id),
  term_id uuid NOT NULL REFERENCES terms(id),
  part_of_term_id uuid REFERENCES parts_of_term(id),
  section_number text NOT NULL,
  delivery_mode text NOT NULL,
  capacity integer NOT NULL CHECK (capacity > 0),
  workflow_state text NOT NULL CHECK (
    workflow_state IN (
      'draft',
      'department_submitted',
      'returned',
      'registrar_review',
      'room_pending',
      'approved',
      'published',
      'change_requested',
      'locked'
    )
  ),
  owning_org_unit_id uuid NOT NULL,
  UNIQUE(course_id, term_id, section_number)
);

CREATE TABLE rooms (
  id uuid PRIMARY KEY,
  building_code text NOT NULL,
  room_code text NOT NULL,
  name text NOT NULL,
  capacity integer NOT NULL CHECK (capacity > 0),
  room_pool_type text NOT NULL CHECK (room_pool_type IN ('central','department')),
  owning_org_unit_id uuid,
  is_active boolean NOT NULL DEFAULT true,
  UNIQUE(building_code, room_code)
);

CREATE TABLE room_features (
  room_id uuid NOT NULL REFERENCES rooms(id),
  feature_code text NOT NULL,
  feature_value text,
  PRIMARY KEY (room_id, feature_code)
);

CREATE TABLE section_meetings (
  id uuid PRIMARY KEY,
  section_id uuid NOT NULL REFERENCES sections(id),
  time_pattern_id uuid REFERENCES time_patterns(id),
  days_mask text NOT NULL,
  start_time time NOT NULL,
  end_time time NOT NULL,
  room_id uuid REFERENCES rooms(id),
  meeting_type text NOT NULL DEFAULT 'primary'
);

CREATE TABLE schedule_change_requests (
  id uuid PRIMARY KEY,
```

```
  section_id uuid NOT NULL REFERENCES sections(id),
  requested_by_user_id uuid NOT NULL,
  request_type text NOT NULL,
  reason_code text,
  justification text,
  status text NOT NULL CHECK (status IN ('draft','submitted','under_review','approve
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Data model notes

- `terms` and `parts_of_term` must be separate because summer and modular offerings often use sub-sessions. [9]
- `time_patterns` should be explicit because many registrars enforce approved scheduling blocks for central classrooms. [2] [1] [9]
- `rooms` should distinguish central from department-controlled inventory because room-governance rules differ. [10] [9]
- `schedule_change_requests` should exist even if edits eventually update `sections`, because post-publish changes commonly require review queues. [3]

Department scheduling workspace

Department schedulers need a **build-and-validate** workspace, not just a table editor. Registrar systems commonly give designated schedulers tools to enter, validate, approve, and update offerings. [2]

Screen layout

```
┌──────────────────────────────────────────────────────────────────────┐
| Semester Institution OS / Department Scheduling                      |
| Term: Spring 2027   Org: College of Arts > History   Status: Open    |
├──────────────────────────────────────────────────────────────────────┤
| Tabs: Sections | Grid Planner | Room Seek | Conflicts | Requests     |
├──────────────────────────────────────────────────────────────────────┤
| Filters                 | Main workspace                             |
|-------------------------|--------------------------------------------|
| Subject                 | [Toolbar: Add section | Submit | Export]   |
| Course level            |                                            |
| Delivery mode           | HIST 101  001  MWF 10:00-10:50  Pending    |
| Instructor              | HIST 210  002  TR 13:00-14:15  Conflict    |
| Room pool               | HIST 325  001  Online         Draft        |
| Conflict only           |                                            |
| Changed only            | Inline warnings and badges                 |
|                         |                                            |
├─────────────────────────┼────────────────────────────────────────────┤
| Validation pane         | Detail panel                               |
| - Time pattern issue    | Section detail                             |
| - Room over capacity    | Meetings                                   |
| - Instructor overlap    | Requested room / room alternatives         |
```

```
| - Missing approval     | Comments and workflow history              |
└──────────────────────────────────────────────────────────────────┘
```

Core views

View	Purpose
Sections table	Bulk editing, state tracking, filters, imports
Grid planner	Visual weekly schedule by day/time
Room seek	Search rooms by capacity, features, and availability
Conflicts	Prioritized conflict resolution queue
Requests	Outbound approval and change-request tracking

Interaction requirements

- Inline validation as users edit.
- Bulk duplicate/copy from prior term.
- Compare against prior published term.
- Show standard time-pattern compliance.
- Suggest alternative rooms/times.
- Warn when instructor or room conflicts emerge.
- Lock approved/published fields appropriately.
- Preserve comments between department and registrar.
- Allow draft save without submission.
- Track unsaved edits and revision history.

Conflict detection is essential because modern scheduling systems are expected to flag overlapping faculty assignments, double-booked rooms, and related issues early in the process. [4]

Detail drawer

A section detail drawer should include:

- Catalog identity.
- Enrollment cap and projected demand.
- Meeting pattern editor.
- Room request and alternatives.
- Delivery mode and special approvals.
- Instructor assignments.
- Notes from registrar.

- Workflow history.
- Submit/withdraw request actions.

Recommended approval state machines

Section approval

```
Draft
  → Department Submitted
  → Registrar Review
  → Room Pending
  → Approved
  → Published
  → Change Requested
  → Registrar Review
  → Approved / Returned / Locked
```

Calendar event approval

```
Draft
  → Pending Review
  → Approved
  → Published
  → Revised
  → Pending Review
  → Approved / Rejected
```

Room request approval

```
Requested
  → Auto-matched / Manual review
  → Conflict
  → Alternative suggested
  → Assigned
  → Confirmed
  → Reassigned / Cancelled
```

Governance requirements

Because this dashboard deals with official schedule and student-related records, governance should be embedded into the workflow.

Required controls:

- Scoped role permissions by campus, college, department, and room pool.
- Justification on overrides and post-publish changes.
- Audit logs for section access, edits, approvals, exports, and publication.

- Published snapshots for official records.
- Dual control for high-risk actions like term publication or broad schedule rollback.
- Read-only auditor access for compliance reviews.

FERPA-aligned systems should use access controls and logging to ensure only appropriate institutional actors handle student-related information. [6] [7]

Build priorities

1. Term and calendar management.
2. Department scheduling workspace with validation.
3. Room inventory and room-seek tooling.
4. Registrar review and publication workflow.
5. Permission engine and audit explorer.
6. Advanced forecasting and demand-aware planning.

That sequence mirrors how institutions typically build schedules: define terms and rules first, let departments build, resolve conflicts, assign rooms, then publish. [1] [2]

⁂

1. https://www.registrar.psu.edu/faculty-staff/scheduling/class-scheduling/
2. https://registrar.calpoly.edu/academic-scheduling
3. https://registrar.utah.edu/scheduling/classes/scheduler-onboarding.php
4. https://moderncampus.com/blog/streamlining-course-scheduling-for-universities-challenges-and-solutions.html
5. https://bluepes.com/blog/role-based-access-education-systems
6. https://openeducat.org/compliance/ferpa/
7. https://studentprivacy.ed.gov/ferpa
8. https://concept3d.com/blog/higher-ed/hidden-cost-broken-room-scheduling-what-universities-can-do-about-it/
9. https://registrar.uiowa.edu/faculty-and-staff/classroom-scheduling/course-scheduling-policies-and-procedures
10. https://registrar.vanderbilt.edu/faculty-staff/academic-scheduling.php
11. https://it.osu.edu/student-information-system-sis/sis-room-class-scheduling/1-course-class-workflow